CS 454/554 — Midterm Mini-Project

One Service, Two Deployment Environments

Due: See Canvas.
Points: 100
Scope: A small project completed over several days, replacing the midterm exam.

Overview

Downloadable copies of these instructions (Markdown, light PDF, and dark PDF) are available in this module. All versions contain the same requirements.

Deploy one small HTTP service in two environments: Docker Compose running inside a Linux virtual machine, and a local Kubernetes cluster. Observe how each environment manages the service, investigate a controlled failure, and explain how you would deploy the same application using OpenStack.

The emphasis is on deploying, observing, and explaining a working system. Keep the application simple. You do not need a database, authentication, a graphical interface, or a public cloud account. You are not required to install OpenStack.

Starter code: cs454-midterm-starter on GitHub. The starter includes the Node.js HTTP service, tests, and deployment planning prompts. Clone or copy it into your own repository. **Do not submit changes to the instructor's course repository.** You must create your own Dockerfile, Compose configuration, and Kubernetes manifests. An equivalent service is acceptable.

Learning objectives

By completing this project, you should be able to:

Explain the relationship between a physical host, a virtual machine, a container, and an application process.

Package and configure a service so that the same container image can run in multiple environments. Compare Compose service management with Kubernetes scheduling, replication, and recovery. Use status information, logs, and HTTP responses to diagnose a deployment problem.

Map a workload to OpenStack compute, image, identity, and networking resources.

Part 1 — Small web service

Use the supplied starter service or an equivalent implementation. The service must provide:

Endpoint	Required behavior
`GET /`	Return JSON containing a configurable service name or message, the serving instance's hostname, and a request count local to that application process.
`GET /health`	Return HTTP 200 while the application is ready to handle requests.

The listening port and service name or message must be configurable using environment variables. Write basic request or startup information to standard output so that it appears in container logs.
The counter is intentionally temporary and local to each process. It may reset when the process restarts, and different replicas may

show different counts. Do not add shared storage to synchronize it. The supplied starter increments the count on successful `GET /` requests (not health checks); if you use an equivalent implementation, describe its counting behavior.

Create a Dockerfile that builds a runnable image. Use this same application image in both deployment environments; document how the image becomes available to your local Kubernetes cluster.

Part 2 — Docker Compose inside a Linux VM

Use an existing Linux VM or create a small one using a virtualization tool appropriate for your computer. Run Docker Engine and Docker Compose inside that VM. The host-to-VM access requirement can use bridged networking, NAT port forwarding, or another documented local route; a public IP is not required.

1. Create a Compose configuration for the web service, including environment variables and a published HTTP port. 2.

   Start the service and demonstrate successful requests to both endpoints from inside the VM.

3. Demonstrate a request from the host computer to the service running inside the VM. Document the VM networking mode and any port forwarding used.
4. Show the service status and container logs.
5. Restart the service and observe what happens to the process-local request count.

Include a small diagram showing the host, VM, container, application, and the path taken by a request from the host. Label relevant ports and addresses. A simple diagram is sufficient.

Part 3 — Kubernetes deployment

Use a local Kubernetes environment, such as the environment used in the course exercises. A second VM or a multi-node cluster is not required. State where your cluster runs relative to your host and VM. You may run Kubernetes on the host or in a suitable local VM, as long as the deployment and observations are documented.

1. Create a Deployment using the same application image, configured with two replicas.
2. Create a Service that selects the application Pods and exposes their HTTP port within the cluster.
3. Demonstrate successful requests through the Service. A documented local access method such as port forwarding is acceptable.
4. Show the Deployment, Pods, and Service, including evidence that two replicas are ready.
5. Delete one application Pod while the Deployment still specifies two replicas. Observe and record the replacement Pod and the return to two ready replicas.

Explain why Kubernetes creates a replacement Pod. Compare this behavior with what you observed when restarting the Compose service.

You do not need to prove round-robin traffic distribution. Some access methods, including port forwarding, can keep requests directed to a single Pod. Explain what your chosen access method actually demonstrates.

Part 4 — Controlled troubleshooting

After the Kubernetes deployment works, intentionally change the Service selector so that it no longer matches the application Pods' labels. Keep the Deployment itself healthy.

Investigate the resulting loss of Service connectivity, then restore the correct selector and verify recovery. Use a request path that depends on the Service's selector; a direct request to a Pod does not test this failure. If your access method establishes a

persistent forwarding session, stop and re-establish that session after changing the selector.
Record:

- The working baseline and the exact configuration change that introduced the failure.
- The symptom observed by a client.
- At least two relevant observations, such as Pod readiness, Pod labels, the Service selector, EndpointSlices, forwarding errors, or application logs.
- Your explanation of the cause, the correction, and a successful request after recovery.

Explain why healthy application Pods do not, by themselves, prove that a client can reach the application through a Service.

Part 5 — OpenStack deployment analysis

Write approximately **500–750 words** explaining how you would deploy this workload on an instructor-provided OpenStack cloud. This is a concrete deployment plan, not an OpenStack installation assignment. Use course material and cite any additional sources.

Address the following:

1. **Resources and choices.** Identify the image, flavor, key pair, tenant network/subnet, router connection to an external network, security group, and floating IP you would need for external access. Justify a modest VM size. State which resources you would create and which might already be supplied by the cloud administrator. Explain whether persistent block storage is needed for this particular workload.
2. **Service responsibilities.** Explain the roles of Keystone, Glance, Nova, and Neutron in authenticating the user, selecting an image, creating the instance, and providing connectivity. Explain where Docker Compose fits once the VM exists. If you propose a Cinder volume, explain its purpose.
3. **Request path.** Trace an HTTP request from an external client through the floating IP and relevant networking controls to the VM, published container port, and application. Identify the security-group rule required for the application port; distinguish it from SSH access and optional ICMP/ping access.
4. **Failure analysis.** Assume the instance is running, but the external client cannot reach the service. Describe a logical sequence of checks that separates application/container problems from VM and cloud-network problems. Identify what evidence you would collect at each layer.
5. **Operational complexity.** Explain why operating an OpenStack cloud involves more components and dependencies than deploying an application into an already working cloud. Use one example from the class demonstration or troubleshooting discussion, and distinguish an infrastructure setup problem from an application deployment problem.

Finish with a brief comparison: when would a VM with Compose be sufficient, and what additional behavior did Kubernetes provide in your experiment?

Additional requirements for CS 554

Complete all requirements above, plus:

- Add CPU and memory requests and limits to the Kubernetes application container. Explain your choices and distinguish a request from a limit.
- Add a readiness probe using `/health`. Explain how readiness affects Service traffic and how it differs from restarting a container.

Perform a short readiness experiment in which **one** replica becomes NotReady while its application process continues running. One permitted approach is to temporarily alter the readiness probe on one Pod so it checks a non-serving path or port. Because Pods managed by a Deployment are recreated from its template, use a temporary per-Pod change (for example, a patch to one running Pod's readiness probe when supported by your Kubernetes environment) or a clearly documented application-level readiness toggle. Do **not** modify the shared Deployment template in a way that makes both replicas NotReady. Do not simply crash or delete the Pod: this is a readiness experiment, not a restart experiment.

Capture a baseline (both Pods Ready and Service endpoints), the failure (one Pod running but NotReady and excluded from ready Service endpoints), and recovery (both Ready and eligible again). Include commands/configuration changes, Pod status, and EndpointSlice observations. If testing HTTP traffic, use an in-cluster client or a method that routes through ready Service backends. Port forwarding to a single Pod, including some Service port-forward sessions, does not establish traffic eligibility for all replicas. Restore any temporary changes after the experiment.

Add approximately **200–300 words** interpreting your resource settings and readiness experiment. This extension is assessed within the Kubernetes and troubleshooting categories below; the total remains 100 points.

Submission

Preferred: submit a GitHub repository URL through the Canvas assignment's Website URL option, or use Text Entry to provide the repository link and a few lines identifying the main files. Use **your own repository**, not a pull request or commit to the instructor's course repository. The repository may be public or accessible to the instructor/grader; verify access before the deadline. Include the report and evidence in the repository as Markdown, PDF, or clearly organized files.

Alternative: Canvas File Upload is acceptable if you cannot share a repository. Upload the relevant source/configuration, documentation, and evidence files. A ZIP archive is **not required** (and is not preferred). If Canvas's file-upload interface requires packaging multiple files, a ZIP is acceptable as a fallback.

Your submission must make these items available:

Application source, Dockerfile, Compose configuration, and Kubernetes manifests.

A README with prerequisites, build/start/access commands for both environments, image-loading instructions, and cleanup commands. State any manual setup steps and name the tools used.

A short technical report with the deployment diagram, VM/Compose versus Kubernetes comparison, and OpenStack analysis. Aim for approximately two pages for the undergraduate report **in addition to** the required 500–750-word OpenStack discussion if needed for completeness; evidence and references do not count toward the suggested page target. Graduate students may use additional space for their 200–300-word extension.

An evidence log with concise, labeled terminal output or screenshots showing successful HTTP requests, VM/Compose status, two ready Kubernetes replicas, Pod replacement, and the controlled failure and recovery. Include the graduate readiness evidence when applicable.

A **3–5 minute recorded walkthrough** demonstrating the service in both environments and explaining one troubleshooting finding. Show and explain results rather than narrating installation steps. Include a clearly labeled recording link in your repository README **and** in your Canvas submission note if possible. A link to an accessible unlisted video or cloud-hosted recording is acceptable; alternatively upload the recording in Canvas if supported. Verify that the instructor/grader has access without requesting permission.

The README and evidence must be sufficient to reproduce and evaluate your deployment **without the instructor needing access to your running VM or cluster**. Do not include VM disk images, exported VMs, container image archives, dependency directories, private keys, passwords, tokens, or cloud credentials. **The deadline and late policy are those shown in Canvas.**

Grading

Category	Points	What is assessed
VM and Docker Compose	25	Working service and image; configurable deployment; host-to-VM access; status/log evidence; correct explanation of layers.
Kubernetes	25	Working Deployment and Service; two ready replicas; Pod replacement evidence; correct explanation of observed behavior. CS 554 also includes resource settings and readiness configuration.
Troubleshooting	20	Controlled failure, relevant evidence, accurate diagnosis, correction, and recovery verification. CS 554 also includes the readiness experiment.
OpenStack and IaaS reasoning	20	Concrete resource choices, accurate service responsibilities, request path, layered diagnosis, and operational reasoning.
Reproducibility and communication	10	Complete files, usable README, clear diagram/report/evidence, references, and concise walkthrough.
Total	**100**	

Keep the project small

One tiny service is enough. Use the same application across environments, focus on the required observations, and explain what your evidence supports. Extra endpoints, a frontend, a database, or a full OpenStack installation do not earn additional credit.